Modélisation mathématiques et *Data Science*

Antoine Nongaillard

But Info – IUT – Université de Lille

R5.A.12 – Séance 01

Plan

Synopsis R5.A.12 : Modélisation mathématique

Data Science et modélisation

La création de modèle

Librairie `numpy`

Librairie `matplotlib`

Synopsis

Objectifs du module R5.A.12 :

- Import et nettoyage de données
- Manipulation de données
- Exploration et mesures statistiques
- Visualisation pertinente
- Modélisation : par régression, par apprentissage

Organisation du module :

- 1h amphi / semaine
- 3h tp / semaine
- qcm / CTPs

Plan

Synopsis R5.A.12 : Modélisation mathématique

***Data Science* et modélisation**

La création de modèle

Librairie `numpy`

Librairie `matplotlib`

La data-science, qu'est-ce ?

Le besoin d'un *data scientist* est apparu pour trois raisons principales :

- l'explosion de la quantité de données produites et collectées par les humains ;
- l'amélioration et l'accessibilité plus grande des algorithmes de traitement des données ;
- l'augmentation exponentielle des capacités de calcul des ordinateurs.

L'objectif est de récupérer des données de plusieurs sources différentes et d'en extraire des informations qui vont servir l'entreprise, notamment pour l'aide à la décision.

Deux composantes sont nécessaires avant de se demander si la *data science* peut, oui ou non, apporter de la valeur et aider à la résolution d'un problème : **des données** et **une problématique bien définie**.

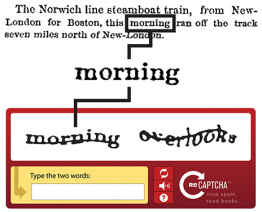
Qu'appelle-t-on *data* ?

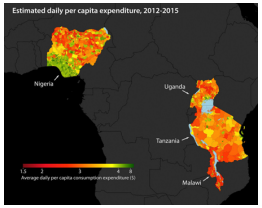
C'est le nerf de la guerre en data science.

- **les bases de données**
  Principales sources de récupération. Bien structurées et assez facile d'utilisation : logs d'un serveur web, catalogue d'un site e-commerce, transactions bancaires...
- **les données brutes**
  Nécessitent des pré-traitements pour les rendre manipulable. Nettoyage important souvent nécessaire pour les rendre exploitable.
- **le texte**
  Texte libre et rédigé en langage naturel. Nécessite un traitement particulier pour être exploitable (*Natural Language Processing*) : code HTML, articles de presse, messages...
- **les images et vidéos**
  Nécessitent des pré-traitements pour être classées et exploitées. Elles posent également des problématiques de stockage en raison de leur volume et de temps de traitement.
- **les IoT** (Internet of Things)
  Nouvelles sources de données brutes. La taille des bases est souvent colossale.

Cycle de travail du *data scientist*

1. **Récupération des données**
   incluant des données hétérogènes (image, son, texte...), des données multi-sources (différents bases), voire la création d'un vecteur de récupération.





Par exemple : estimation de la pauvreté via des images aériennes/satellites ou la détection de l'illettrisme via le smartphone (reCAPTCHA).

Cycle de travail du *data scientist*

2. **Nettoyage et regroupement des données**
   rôle du *data architect* qui doit garantir que les données sont consistantes, sans valeurs aberrantes ni manquantes, exprimées dans un même format et accessible à partir d'un même endroit et au bon moment.

3. **Exploration des données**
   rôle du *data analyst* qui doit mieux comprendre les différents comportements afin de saisir le phénomène sous-jacent. À la fin de l'exploration, vous devrez être en mesure de :
   - proposer plusieurs hypothèses sur les causes sous-jacentes à la génération du jeu de données (ex : "*il y a clairement une relation entre X et Y*")
   - proposer plusieurs pistes de modélisation statistique des données, qui vont permettre de résoudre la problématique de départ
   - proposer si nécessaire de nouvelles sources de données qui aideraient à mieux comprendre le phénomène.

Cycle de travail du *data scientist*

4. **Modélisation des données**
   rôle spécifique du *data scientist* : utilisation d'algorithmes pour créer de l'intelligence (artificielle) ou de la connaissance, qui aide à la décision.

   L'objectif est de trouver un modèle (stochastique ou déterministe) du phénomène à l'origine des données. C'est-à-dire qu'on considère que chaque donnée observée est l'expression d'une variable aléatoire générée par une distribution de probabilités.

5. **Exploitation du modèle**
   Une fois le modèle éprouvé, on l'utilise pour prendre des décisions plus pertinentes.

La *data science*, à quoi ça sert ?

Il existe un spectre assez large de problématiques que l'on peut résoudre en data science :

- **prédire la rentabilité d'une campagne marketing**
  Les entreprises ont souvent besoin d'évaluer le ROI (retour sur investissement) d'une campagne (ou d'en comparer plusieurs) avant de la lancer.
- **recommander un produit à un client**
  C'est suggérer d'autres produits à acheter sur Amazon, des films à regarder sur Netflix, des musiques à écouter sur Spotify. . .c'est une proposition de contenu similaire aux produits qu'a déjà aimé l'utilisateur.
- **segmenter les visiteurs d'un site**
  À partir d'une connaissance qualitative de la clientèle (âge, genre, ville. . .), vous souhaitez catégoriser automatiquement vos clients selon la probabilité qu'un individu réponde à une sollicitation marketing ou appartienne à la cible recherchée.
- **identifier les événements rares**
  C'est un type de catégorisation automatique pour identifier les spams d'e-mail, les transactions frauduleuses, les textes injurieux. . .

Plan

Synopsis R5.A.12 : Modélisation mathématique

Data Science et modélisation

La création de modèle

Librairie `numpy`

Librairie `matplotlib`

Un modèle, à quoi ça ressemble ?

Un modèle peut prendre différentes formes selon :

- les données disponibles : annotées (comme dans les banques d'images) ou non
- la nature des variables :
  - qualitative nominale : *la couleur des cheveux*
  - qualitative ordinale : *des mentions à un examen*
  - quantitative discrète : *un nombre de places*
  - quantitative continue : *la taille d'une personne*
- le type de résultat attendus :
  - résultat qualitatif : *une catégorie*
  - résultat quantitatif : *une probabilité de succès*

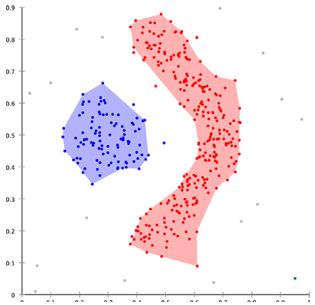
Comment caractérise-t-on une date ?

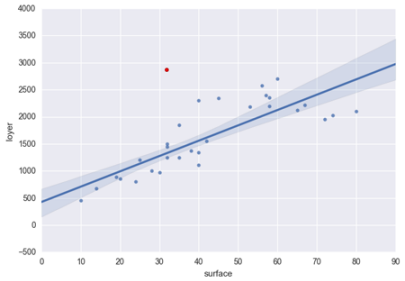
À quoi ça ressemble ?

Deux exemples différents de représentation :

- l'un pour faire de la classification
- l'autre pour faire de la régression

Lequel pour faire quoi ?

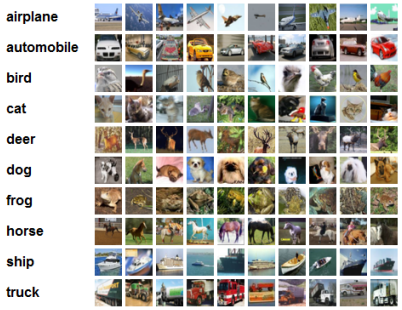




Les types d'apprentissage

L'**apprentissage supervisé (*supervised learning*)** se base sur des données annotées de leur sortie (parfois, manuellement), pour entraîner le modèle. Le modèle, une fois entraîné, doit prédire cette sortie sur de nouvelles données non annotées.

On reçoit des données annotées $(x_1, y_1), (x_2, y_2), \ldots$ et on espère prédire la sortie sur de nouvelles observations : $x^\star \to y^\star$.



Les types d'apprentissage

En **apprentissage non supervisé (*unsupervised learning*)** les données d'entrée ne sont pas annotées. L'algorithme s'applique à trouver seul les similarités et les distinctions au sein des données d'entrée.

À partir d'observations brutes de variables aléatoires $x_1, x_2, \ldots$, on espère découvrir la relation avec des variables latentes structurelles :

$$x_i \to y_i$$

Les algorithmes

Beaucoup d'algorithmes existent et se caractérisent différemment au sein de deux familles :

- l'apprentissage (semi-)supervisé qui nécessitent des données annotées
- l'apprentissage par renforcement qui se base sur un cycle d'expérience/récompense afin d'améliorer ses performances à chaque cycle.
- paramétrique ou non, génératif ou discriminatif...

Voici certains algorithmes :

- régression linéaire, polynomiale ou logistique
- K-nearest neighbor (KNN), Support Vector Machine (SVM)
- réseaux de neurones, DQN
- arbres de décision, random forest

Mesurer les performances fait partie intégrante du travail de modélisation. Une mauvaise mesure de performance (faux-positif, faux-négatif) garantit un mauvais apprentissage et de mauvaise prédiction !

Les outils

Les langages :

- `python` plutôt pour les informaticiens
- `R` pour les statisticiens
- `sql` dès qu'il y a une base de données

Les librairies `python` :

- pour la récupération des données : `scrapy` (texte page web)
- pour le nettoyage des données : `numpy`, `pandas`, `scipy`
- pour la visualisation : `matplotlib`, `seaborn`
- pour la modélisation et l'apprentissage : `scikit-learn`, `pytorch`, `tensorflow`, `keras`

Tout ça dans des feuilles `Jupyter` !

Plan

Synopsis R5.A.12 : Modélisation mathématique

Data Science et modélisation

La création de modèle

Librairie `numpy`

Librairie `matplotlib`

Motivations

`numpy`, signifiant *Numerical Python*, vient du constat de l'inefficacité incroyable de python pour les calculs.

Les listes python peuvent être hétérogènes : gestion mémoire inefficace, algorithme de manipulation très générique.

Les `ndarrays` de la librairie `numpy` sont des listes enrichies (de dimension `n`), optimisées pour le calcul.

- tableau homogène
- allocation mémoire fixe et prédictible
- consommation mémoire réduite
- exécution jusqu'à 30 fois plus rapide

Création de tableau

Par convention, on écrit l'import de la librairie de la manière suivante :

```
import numpy as np
```

- Création d'un `array` à partir d'une liste donnée :

```
tab = np.array([x1, x2, ...])
tab = np.array([[x11, x12, ...], [x21, x22, ...], ..., [xn1, xn2, ...]])
```

- Création d'un `array` à partir d'un fichier :

```
tab = np.genfromtxt('filename.txt', delimiter=';', skip_header=True)
```

- Génération d'un `array` à partir d'une fonction :

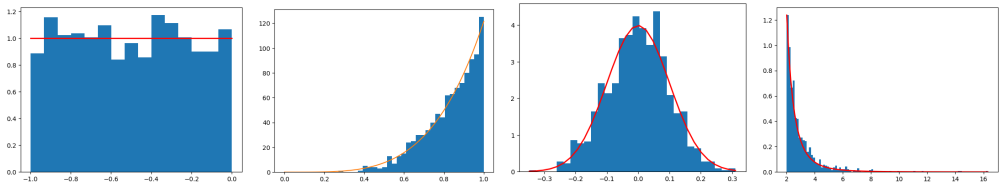
```
tab = np.zeros((4,6))
tab = np.ones((3,7), dtype=int)
tab = np.emptys((2,3), dtype=int) # pour une génération aléatoire
tab = np.arange(2,9,3) # séquence linéaire de 2 à j9 par pas de 3
tab = np.linspace(1,7,5) # renvoie un tableau de 5 valeurs espacées régulièrement entre 1 et 7
```

Création de tableau

`numpy` est doté d'un module `random` très complet. On commence toujours par créer un générateur :

```
gen = np.random.default_rng()
```

Sur ce générateur, on peut invoquer pas moins de 36 distributions de probabilité différentes comme `uniform(...)`, `power(...)`, `normal(...)` ou `Pareto(...)`.



D'autres méthodes de génération sont possibles comme `integers(...)` ou `random(...)` pour les entiers et les réels, `choice(...)` pour créer un échantillon sur-mesure,

Accès aux éléments

- Indexation simple :

```
tab[2]
tab[2,1] # en deux dimensions
```

- Indexation booléenne :

```
tab > 2 # renvoie un tableau booléen de même dimension
tab[tab > 2] # renvoie un tableau (n=1) contenant les éléments
tab[tab[:,0]>0] # ne conserve que les lignes de 1er élément positif
```

- Indexation '*fancy*' :

```
tab[[5,2,8,3]] # renvoie un tableau contenant les valeurs d'indice
5-2-8 et 3.
```

- Accès par tranche (*slicing* `<start:stop:step>`) :

```
tab[:3] # les 3 premiers
tab[10:] # à partir du 10ème
tab[-2:] # les deux derniers
tab[-1,:] # la dernière ligne d'une matrice
```


Autres opérations

- Opérations arithmétiques (attention aux tailles) :

```
np.add(tab1, 6) # ajoute 6 à tous les éléments
np.add(tab1, tab2)
np.substract(tab1, 1) # retire 1 à tous les éléments
np.multiply(tab1, 5) # multiplie tous les éléments par 5
np.divide(tab1, 3) # divise tous les éléments par 3
```

- Opérations d'agrégation :
  `min()`, `max()`, `sum()`, `median()`, `mean()`, `cumsum()` (somme cumulée), `std()` (écart-type), `argmin()` et `argmax()`.
  Ces opérations peuvent s'appliquer sur tous les éléments, par ligne (via l'option `axis=1`) ou par colonne (via l'option `axis=0`).

- Les attributs :

```
tab.dtype # le type des éléments
tab.size # le nombre d'éléments
tab.ndim # la dimension
tab.shape # la taille des dimensions
tab.T # la transposée
```


Ajout, suppression, découpage

- Ajout/insertion d'éléments :

```
np.append(tab, [1,2]) # ajout de 1 et 2 en queue
np.append(tab, [[10, 10]], axis=0) # ajout d'une ligne donnée
np.insert(tab, 0, [11,11]) # ajout des valeurs données en début
```

- Suppression d'éléments :

```
np.delete(tab, 1, axis=1) # colonne d'indice 1 supprimée
np.delete(tab, [1,4], axis=0) # suppression des lignes d'indice 1 et
4
```

- Découpage d'un tableau :

```
np.split(tab, 6) # casse tab à l'indice 6
np.hsplit(tab, 6) # casse tab à la ligne d'indice 6
np.vsplit(tab, 6) # casse tab à la colonne d'indice 6
res1, res2, res3 = np.split(tab, [4,7])
```

Combinaison et calcul matriciel

- Misc, mais important quand même :
```
np.where(tab1>0) # pour filtrer selon une condition donnée
np.count_nonzero(tab1)
np.reshape(tab1, [4,3]) # pour redimensionner ici en 4lx3c
```
- Concaténation (attention aux dimensions) :
```
np.concatenate(tab1, tab2)
np.vstack((tab1, tab2)) # empilement vertical
np.hstack((tab1, tab2)) # empilement horizontal
np.stack((v1,v2)) # empilement paramétrique
np.column_stack((tab1, tab2)) # juxtaposition
```
- Manipulation de matrices :
```
matrice = np.mat(tab) # création d'une matrice
matrice.T # la transposée
matrice.H # la transposée conjuguée
matrice.I # l'inverse
matrice**2 ou matrice.dot(matrice) # pour les produits avec elle-même
```


Plan

Synopsis R5.A.12 : Modélisation mathématique

Data Science et modélisation

La création de modèle

Librairie `numpy`

Librairie `matplotlib`

Motivations

La librairie `matplotlib` est dédiée à la visualisation des données. La finesse de son paramétrage et sa facilité de prise en main l'a rendu très populaire.

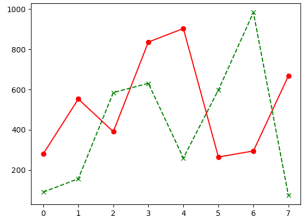
La plupart du temps, l'import du module `pyplot` de la librairie suffit généralement et se fait par l'instruction suivante :
```
import matplotlib.pyplot as plt
```

Il existe 5 types de visualisation **de base** qui ont des usages différents :
- `lineplot`
- `scatterplot`
- `bargraph`
- `histogram`
- `pieplot`

Pour bien choisir, il faut avoir une question précise à laquelle répondre.

lineplot

Tracé de courbes standard, qui permet de représenter l'évolution d'une variable quantitative par rapport à une autre.

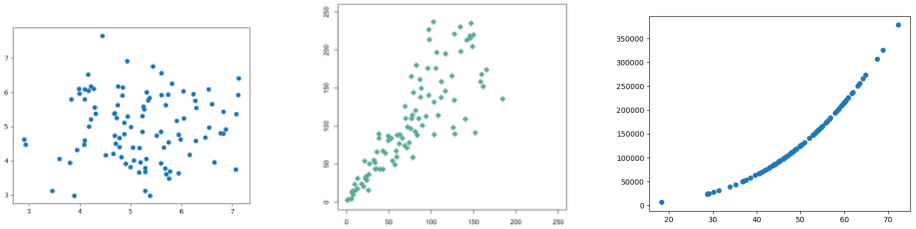


Le code permettant de générer cette figure est le suivant :
```
gen = np.random.default_rng() # définition du générateur aléatoire
x = np.arange(0, taille)
y = gen.integers(0, 1000, size=(taille, 2))

plt.plot(x, y[:,0],  marker='o', color='red')
plt.plot(x, y[:,1], marker='x', color='green', linestyle='--')
plt.show()
```

scatterplot

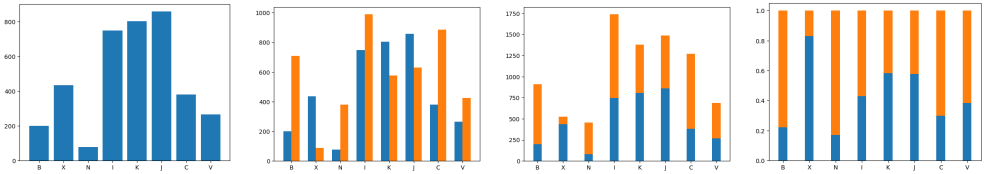
"*Nuage de points*" en français, permet de représenter deux variables quantitatives, pour notamment identifier des relations / corrélations entre elles.



On utilise la méthode `plt.scatter(x, y)` où `x` et `y` décrivent respectivement les abscisses et les ordonnées des points à tracer.

bargraph

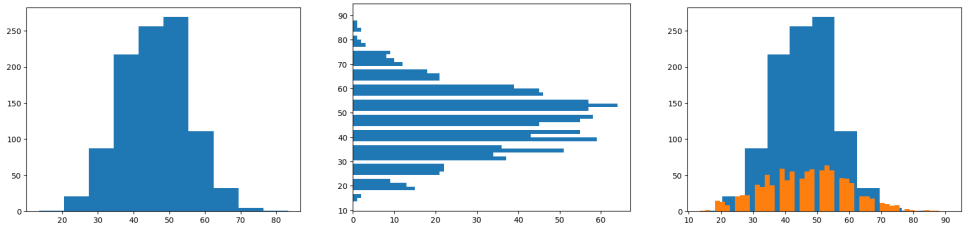
"*Graphique à barres*" en français, permet de représenter des variables qualitatives et les valeurs quantitatives associées.



On utilise la méthode `plt.bar(x, y)` où `x` et `y` décrivent respectivement les modalités et les hauteurs des barres à tracer.

histogram

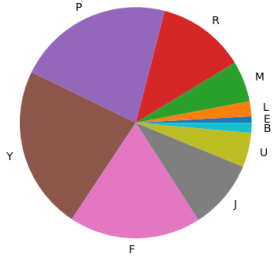
"*Histogramme*" en français, permet de représenter des variables quantitatives et leur répartition selon des '*bins*' de largeur donnée.



On utilise la méthode `plt.hist(x, bins, weights)` où `x` correspond aux valeurs, `bins` aux catégories et `weights` à leur dénombrement dans les catégories.

pieplot

"*Camembert*" en français, cette représentation illustre la répartition relative d'une variable quantitative au regard d'une variable qualitative.



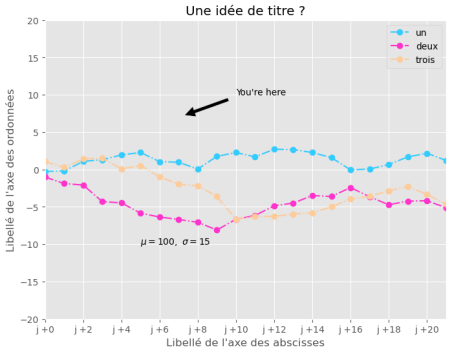
On utilise la méthode `plt.pie(x, labels)` où `x` décrit les valeurs et `labels` les étiquettes à associer à chaque catégorie. Les dénombrement ou regroupement doivent être effectué au préalable.

Personnalisation d'un graphe

`matplotlib` est hautement personnalisable. Un certain nombre d'instruction permettent d'enrichir un graphe, toutes sont disponibles avec beaucoup d'option.

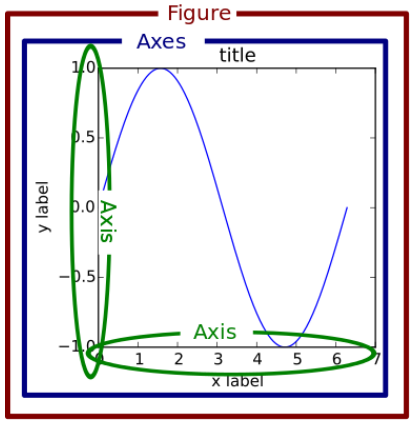
Par exemple :

- `title(...)`
- `xlabel(...)` et `ylabel(...)`
- `xticks(...)` et `yticks(...)`
- `xlim(...)` et `ylim(...)`
- `legend(...)`



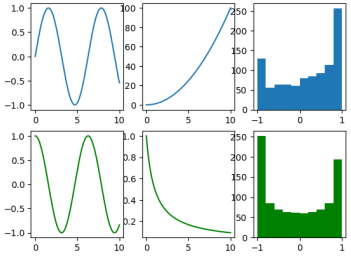
La structure d'un graphe

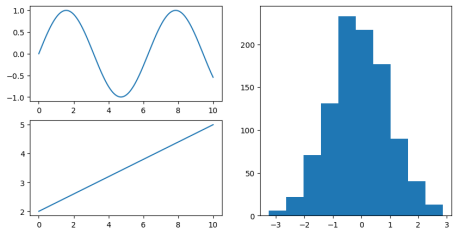
On peut personnaliser tout ce qu'on veut tant qu'on a compris la structure d'un graphe :



Plusieurs graphes ?

Il est également possible d'intégrer plusieurs graphes à une même image :





```
fig, axes = plt.subplots(2, 3)
axes[0, 0].plot(time, height)
axes[0, 1].plot(time, time**2)
axes[0, 2].hist(height)

axes[1, 0].plot(time, weight, color='green')
axes[1, 1].plot(time, 1/(time+1), color='green')
axes[1, 2].hist(weight, color='green')
```


Exemples

